

FinPilot

Personal Finance Management System for Android

Technical Design Document — Assignment 01, 2026 Semester 01

Document Type	Technical Design Document
Module	SE3092 — Platform Based Development
Assignment	Assignment 01 — Group Assignment
Submission Deadline	24 May 2026
Technology Stack	Kotlin · Jetpack Compose · MVVM · Firebase
Team Members	Kaveen Venuja Vinuki Sahaji

Table of Contents

1 Introduction	3
2 Phase One — Problem Analysis	4
2.1 Income Structure Analysis	4
2.2 UX Failure Analysis — Why Previous Tools Failed	5
2.3 Goal-Tracking Analysis	7
2.4 Expense-Entry Friction Analysis	7
2.5 Spending Visibility Analysis	8
2.6 Derived Insights	8
3 Requirements Engineering	9
3.1 Functional Requirements	9
3.2 Non-Functional Requirements	10
3.3 Constraints	11
4 System Design	12
4.1 Architecture Overview	12
4.2 MVVM Architecture Diagram	12
4.3 Navigation Flow	13
4.4 Repository Pattern	13
5 Database Design	14
5.1 Firestore Schema	14
5.2 Collections & Documents	15
5.3 Multi-Currency Handling	16
5.4 Indexing Strategy	17
5.5 Firestore Security Rules	17
6 UI/UX Design — Annotated Wireframes	18
7 Design Justifications	22
8 Conclusion	24

1. Introduction

1.1 Project Overview

FinPilot is a native Android personal finance management application built for SE3092 Platform Based Development, Assignment 01. It is designed to solve a concrete, well-documented problem: that of a young Sri Lankan software engineer who earns well from multiple income channels, spends impulsively, and has repeatedly failed to maintain any financial visibility despite genuine effort. FinPilot is not a generic budgeting app retrofitted onto this user — it is a system built from the ground up around Kavindu Silva's specific combination of complexity, irregularity, and aspiration.

Component	Technology / Choice
Language	Kotlin 100% — no Java in application layer
UI Framework	Jetpack Compose with Material Design 3
Architecture	MVVM — strict layer separation
Navigation	Navigation Compose (single-activity)
State Management	StateFlow / collectAsState()
Async	Kotlin Coroutines + viewModelScope
Authentication	Firebase Authentication (email/password + Google Sign-In)
Primary Database	Firebase Firestore (real-time listeners)
Local Cache	Room (offline persistence)
Dependency Injection	Hilt
Exchange Rates	OkHttp + OpenExchangeRates API
Charts	YCharts (co.yml:ycharts:2.1.0)
Min SDK	API 26 (Android 8.0 Oreo)
Target SDK	API 36

1.2 Understanding Kavindu's Problem

The assignment provides not a feature list but a scenario. Kavindu Silva, 25, is a junior software engineer at a Colombo startup. He earns LKR 120,000–140,000/month in salary, plus freelance project income (LKR 20,000–90,000 per project), Google AdSense revenue in USD, and cryptocurrency trading gains/losses in USDT/ETH. After 12 months of earning over LKR 1.6 million, he has LKR 11,200 in savings. He cannot explain where the rest went. Every prior tracking attempt has failed within two weeks.

His financial goal — a MacBook Pro M4 at ~LKR 490,000 — is emotionally present but financially invisible. No tool has ever connected his daily spending decisions to the timeline of that goal. FinPilot was designed to change that.

1.3 Team Composition

Member	Phase 1 Section	Implementation Focus
--------	-----------------	----------------------

Kaveen	Section 1 (Income Analysis) & Section 5 (System Summary)	Architecture, Firestore schema, DI, Repositories
Venuja	Section 2 (Data Model & Schema)	Database design, Room, FinanceRepository
Vinuki	Section 3 (UX Failure Analysis)	UI screens, Jetpack Compose, UX flows
Sahaji	Section 4 (Goal & Visualisation)	Goal screen, dashboard charts, visualisation logic

2. Phase One — Problem Analysis

This section carries 20 marks (20% of the total grade). It demonstrates that we understood the problem deeply enough to design a system that would genuinely change Kavindu's relationship with his money — not merely produce functional screens.

2.1 Income Structure Analysis

Kavindu's most fundamental problem is that his income is structurally irregular. He earns from four distinct channels, each with a different currency, payment frequency, certainty level, and psychological weight. Any system that models income as a single monthly figure will fail him, precisely as every previous tool has.

Income Source	Currency	Frequency	Range (LKR)	Certainty	Core Challenge
Monthly Salary	LKR	Monthly (25th)	120,000–140,000	High	Exact amount varies with bonus; payslip rarely reviewed
Freelance Projects	LKR	Milestone-based	20,000–90,000/project	Low	Irregular timing; invoices forgotten; project-payment reconciliation fails
Google AdSense	USD → LKR	Monthly	~3,500–19,000 equiv.	Medium	FX rate at transfer varies; never formally recorded
Crypto Trading	USDT/ETH	Ongoing	Variable gain/loss	Very Low	No running P&L; net position unknown; losses psychologically unacknowledged

Key Insight

The system must treat income as a stream of discrete events — each with its own source, currency, receipt date, and LKR equivalent — not as a single monthly constant. Only by recording what actually arrived, when, and in what currency can FinPilot give Kavindu an accurate picture of what he truly earns. His self-estimate of 'LKR 160,000–210,000' has never been validated. FinPilot makes the actual versus estimated gap visible.

2.1.1 Salary Variance Handling

Salary is not stored as a fixed monthly constant. FinPilot records each month's actual salary received. An optional base salary estimate may be stored for projection, but all analytics use actual figures. This prevents the systematic overestimation that has made Kavindu's income calculations unreliable.

2.1.2 Freelance Project Layer

Freelance income requires a lightweight project layer. When recording a freelance payment, Kavindu can tag it to a named project (e.g. 'React build for Cinnamon Hotel'). This serves two purposes: it prevents the reconciliation confusion he has experienced (which payment belongs to which project), and it enables the system to show outstanding vs received amounts per project over time. The `FreelanceProject` entity stores `clientName`, `projectTitle`, `agreedAmount`, `paidAmount`, `status`, and a list of income entry IDs.

2.1.3 Foreign Currency Normalisation Strategy

All income is expressible in LKR for aggregate reporting, while preserving the original currency value. Each income record stores both the foreign currency amount (`amountOriginal` + `currencyOriginal`) and the LKR equivalent (`amountLKR`) computed at entry time using a user-supplied exchange rate. This is more honest than assuming a fixed rate and implicitly maintains an exchange rate audit trail. Aggregation queries sum `amountLKR` only — no runtime currency conversion required.

2.1.4 Crypto: Realised vs Unrealised Income

FinPilot records crypto income only when Kavindu withdraws or converts to fiat — the realised event. Unrealised portfolio value does not inflate apparent income. He may log trades for reference, but only withdrawal events count toward monthly income totals. This is the distinction Kavindu has never had in any tool — his Q3 loss remained unacknowledged because it was never separated from the Q1 gain.

2.2 UX Failure Analysis — Why Previous Tools Failed

Kavindu has failed with four distinct tools. The assignment identifies this as central to the design challenge. These are not independent failures — they share a consistent set of behavioural and UX-level root causes that FinPilot is specifically designed to prevent.

Tool	Duration Used	Proximate Failure	Underlying UX Failure
Google Sheets	~13 days	Formulas broke after row insertion	High maintenance cost per entry; no mobile-first experience; zero tolerance for imperfect data
Play Store Budget App	9 days	USD only, no cloud sync, single income model	Wrong data model; no LKR support; architecture assumed one salary source in one currency
Notes App (bank export)	~4 days	Became unreadable block of figures	No structure, no categorisation, no visual summary; cognitive overload from raw text
Bank Native Summary	Ongoing (passive)	Card-only; miscategorised PickMe as 'online retail'	Incomplete data model; no user control; false categorisation erodes trust in the system

2.2.1 Root Cause 1 — High Friction Data Entry

Every tool required significant manual effort to enter data accurately. Sheets required typing amounts, dates, and categories into cells. The notes app required pasting and reformatting raw exports. High friction means that any interruption — a freelance deadline, a busy week — becomes permanent abandonment. When Kavindu returned to his Sheet after three weeks away, the gap felt more discouraging than the tool felt useful.

FinPilot Response	FAB always visible. Quick-add expense completable in 3 taps: Amount → Category → Confirm. Date, payment method, and currency pre-filled to the most common case (today, CARD, LKR). All non-essential fields are optional. No entry is 'incomplete' enough to reject.
--------------------------	---

2.2.2 Root Cause 2 — No Visual Feedback Loop

None of the tools produced a visual that showed Kavindu the relationship between his income, spending, and goal. His bank app showed him a transaction list — not a picture. His Sheet showed him numbers — not meaning. The LKR 24,000 coffee/dining discovery came from a manual 3-month statement review, not from any tool. Humans process visual patterns faster than numerical tables — a pie chart showing 18% of income going to food changes behaviour in a way that the raw number LKR 24,000 does not.

FinPilot Response	Dashboard is chart-first. Spending distribution donut chart, income source bar chart, and goal progress card are all above the fold — visible without scrolling. The dashboard is the reason to open the app daily.
--------------------------	---

2.2.3 Root Cause 3 — Wrong Data Model

The Play Store app assumed a single income source and a fixed monthly budget in USD. This is not an implementation failure — it is a requirements failure. The app was built for a different user archetype. Kavindu's context (multi-source income, LKR with USD side income, impulsive discretionary spending, a specific quantified goal) requires a data model specifically designed for him.

**FinPilot
Response**

Five Firestore collections: users, income, expenses, goals, freelanceProjects. Income model supports 4 source types (SALARY, FREELANCE, ADSENSE, CRYPTO) with per-entry currency selection. Expense model supports isRecurring flag for committed/discretionary split.

2.2.4 Root Cause 4 — No Goal-Behaviour Connection

Kavindu's MacBook goal is 'emotionally present but financially invisible'. None of his tools showed him how today's LKR 2,000 UberEats order affected his MacBook timeline. The absence of this connection removes the motivational feedback loop that sustains consistent behaviour. When saving feels abstract, spending feels consequence-free.

**FinPilot
Response**

Goal card on dashboard updates in real time. Monthly required savings computed as $(\text{targetAmount} - \text{currentAmount}) / \text{monthsRemaining}$ — visible on every app open. Post-expense micro-insight: 'Did you know? [Category] made up X% of your spending this month.'

2.2.5 Root Cause 5 — No Recovery After Gaps

When Kavindu missed entries for a week, all four tools became worse than useless — they showed incomplete data that felt more discouraging than no data. A well-designed system must be forgiving of gaps. Missing a week should not break the system's analytical value for the other three weeks.

**FinPilot
Response**

Month-over-month comparison uses whatever data exists. No punitive empty states. Dashboard always opens to a useful state even with zero entries, showing the goal and prompting the first action positively. Partial data is treated as valid data.

2.3 Goal-Tracking Analysis

Kavindu's MacBook goal is not abstract — it is quantified (LKR 490,000), time-bound (12 months from now), and personally meaningful (a genuine professional need). Yet no tool has connected his daily financial behaviour to this goal. He does not know what monthly savings rate is required. The goal exists in his mind but nowhere in his financial system.

2.3.1 What FinPilot Does Differently

Computed Monthly Requirement: FinPilot computes $(\text{targetAmount} - \text{currentAmount}) / \text{monthsRemaining}$ and displays this prominently. As Kavindu saves more, the required rate drops. As he overspends, the figure updates immediately — creating a real-time feedback loop that has never existed in any of his prior tools.

Explicit Savings Events: The goal's currentAmount is not auto-calculated from expense records. FinPilot treats saving as an explicit act — Kavindu logs a savings event when he deliberately sets money aside. This creates a transparent record of deliberate saving, separate from the organic surplus. Additionally, FinPilot computes a projected surplus (total income – total expenses) as a secondary figure alongside the explicit savings record.

Savings History Chart: A bar chart of monthly savings contributions over the last 6 months, sourced from the `goals/{goalId}/savingsLogs` subcollection, shows whether behaviour is improving, stable, or deteriorating. Months with no entries show LKR 0, keeping the chart full-width and non-broken.

Multiple Goals Support: GoalViewModel supports a pager of multiple goals, each with independent progress tracking. Kavindu can create a new goal without abandoning the MacBook tracker.

2.4 Expense-Entry Friction Analysis

Since FinPilot cannot import bank data automatically, expense entry design is the most critical UX decision in the entire product. The design principle is: speed over completeness — a fast, incomplete entry beats a slow, complete entry that never gets made.

Expense Field	Required?	Default Value	Input Method
Amount	Yes	None (user-entered)	Numeric keypad
Category	Yes (enforced)	OTHER	Single-tap chip selector (8 categories)
Date	No	Today	Pre-filled; date picker on tap
Payment Method	No	CARD	Dropdown (CARD, CASH, BANK_TRANSFER, AUTO_DEBIT)
Note	No	Blank	Optional text field
Recurring Flag	No	False	Single toggle
Sub-Category	No	None	Optional text
Tags	No	Empty	Optional multi-tag

The three-tap flow: (1) Enter amount → (2) Tap category chip → (3) Press 'Add Expense'. Everything else is pre-filled. This is a direct architectural response to Kavindu's Sheet abandonment — the friction is lower than any manual tool he has tried.

2.5 Spending Visibility Analysis

Kavindu has no category system and no concept of what proportion of his income is discretionary versus committed. He has never seen a visual representation of how his spending is distributed. FinPilot provides this

visibility through five key views:

#	Visualisation	What It Reveals	Screen
1	Spending Category Donut Chart	How income is distributed — surfaces the LKR 24,000 coffee discovery instantly	Dashboard
2	Income Source Breakdown Bar	Proportion of income from each source; makes irregular income visible and comparable	Dashboard
3	Goal Progress Card	Distance from LKR 490,000 and required monthly rate; transforms goal from abstract to actionable	Dashboard + Goal Screen
4	Committed vs Discretionary Split	Percentage of income already allocated to fixed costs (isRecurring flag) vs genuinely flexible	Dashboard Insights
5	Monthly Savings Trend	Month-over-month savings history; shows whether behaviour is improving or deteriorating	Goal Screen

2.6 Derived Insights — Summary Table

Kavindu's Pain Point	Root Cause	FinPilot Design Response
Cannot calculate monthly income	4 sources, 3 currencies, no single view	Dashboard income breakdown by source; all normalised to LKR via amountLKR field
Tracking always abandoned within 2 weeks	High friction; no reward for consistent use	3-tap expense entry; chart-first dashboard worth opening daily
No spending category awareness	No structure in any prior tool	Category enforced at entry; donut chart visible on home screen
Goal emotionally present, financially invisible	No tool connected spending to goal timeline	Goal card on dashboard; real-time monthlyRequired computation; savings logs
Cannot reconcile freelance payments	No project-payment linkage model	FreelanceProject entity; income entries tagged with projectRef
App fails after missed entries	All prior tools punish gaps	Month view shows partial data; no punitive empty states; always useful
Multi-currency income not tracked	Tools assumed single currency	Per-entry currencyOriginal + amountLKR + exchangeRate; user-supplied FX rate
Crypto gains/losses unrecognised	No realised vs unrealised distinction	Only withdrawal events count as income; unrealised gains excluded from totals

3. Requirements Engineering

All requirements below are derived directly from the Kavindu Silva scenario. No generic features are listed — every requirement traces to a specific failing identified in Phase One analysis.

3.1 Functional Requirements

ID	Requirement	Scenario Traceability
FR-01	Multi-Source Income Recording: record income from SALARY, FREELANCE, ADSENSE, CRYPTO (and custom) with per-entry currency selection and LKR normalisation	§3.2 Income Problem — 4 sources; §1.4 Income model failure
FR-02	Categorised Expense Entry: record expenses with mandatory category, optional sub-category, payment method, date, note, and recurring flag. Entry must be completable in 3 taps for the common case	§3.3 Expense invisibility; §3.5 Failed attempts — friction
FR-03	Goal Definition & Progress Tracking: define a savings goal with target amount and deadline; compute and display required monthly savings rate; update in real time	§3.4 Goal that does not move; §4.2 Goal/ViewModel
FR-04	Dashboard Financial Summary: present current-month income by source, spending by category (donut chart), goal progress, and committed/discretionary ratio without navigation	§3.3 No category system; §3.6 What solution needs to be
FR-05	Multi-Currency Income: store income in original currency AND LKR equivalent; user-supplied exchange rate at entry time; all aggregation uses amountLKR	§3.2 AdSense in USD; crypto in USDT/ETH
FR-06	Freelance Project Linking: tag freelance income entries to named projects; track agreedAmount vs paidAmount; show outstanding vs received per project	§3.2 Forgotten invoices; reconciliation failure
FR-07	Recurring Expense Identification: mark expenses as recurring (isRecurring flag); compute committed vs discretionary ratio from this flag	§3.3 Gym auto-debit; subscriptions; committed cost opacity
FR-08	Firebase Authentication: email/password (mandatory); Google Sign-In (bonus); email verification; password reset; account deletion	Assignment §4.3 — Firebase Auth required
FR-09	Offline Functionality: app remains functional without network using Room local cache; data syncs to Firestore on reconnection; no blank screens on network failure	Assignment §4.3 — offline persistence required
FR-10	Error State Handling: no crashes on empty datasets; graceful loading states; user-friendly error messages for all Firebase exceptions	Assignment §4.3 — all error states must be handled
FR-11	Month-over-Month Comparison: compare current month income, expenses, and net balance vs previous month; show percentage change	§2.6 — Kavindu has never seen trend data
FR-12	Savings Log History: record individual savings events with timestamp; aggregate into monthly bar chart over 6 months	§3.4 Goal tracking; §4.3 Visualisation requirements

3.2 Non-Functional Requirements

ID	Requirement	Rationale
NFR-01	Minimum SDK API 26 (Android 8.0 Oreo), Target SDK 36	Assignment constraint; ensures broad device compatibility
NFR-02	100% Kotlin — no Java files in application layer	Assignment constraint; language consistency
NFR-03	Jetpack Compose only — no XML layouts accepted	Assignment constraint; modern declarative UI paradigm
NFR-04	Strict MVVM — no business logic in Composables; data layer calls must not originate from UI layer	Maintainability, testability, assignment requirement
NFR-05	StateFlow for all ViewModel state — no raw mutable state exposed to UI layer	Reactive state management; lifecycle safety
NFR-06	Hilt for dependency injection — no manual ViewModel instantiation inside Composables	Reduces boilerplate; improves testability; assignment preference
NFR-07	viewModelScope for all async operations	Assignment constraint; prevents coroutine leaks
NFR-08	App must display first meaningful content within 2 seconds on a modern device	Kavindu's low tolerance for friction — a slow app will be abandoned as quickly as the others
NFR-09	All LKR amounts displayed with thousand separators (e.g. 120,000 not 120000)	Readability of large Sri Lankan Rupee figures; professional presentation
NFR-10	Firestore Security Rules must be submitted as a deliverable	Assignment §4.3; data isolation requirement
NFR-11	Single-activity architecture with Navigation Compose	Assignment constraint; modern Android navigation pattern
NFR-12	Material Design 3 components throughout	Assignment constraint; design consistency

3.3 Constraints

Constraint	Description
No automatic bank import	All income and expense entries are manual. No open banking API is integrated.
No live exchange rate at runtime (v1)	Exchange rates are user-supplied at entry time. The ExchangeRatesRepository provides a reference rate but final LKR equivalent is user-confirmed.
Firebase Spark Plan	Free tier limits apply. No Cloud Functions available. All aggregation occurs client-side.
Single user per account	No multi-user household budgeting. One Firebase Auth account per installation.
English only (v1.0)	No internationalisation in the first release.
Kotlin only	No Java interop in the application layer. Kotlin DSL for Gradle.
Room as local mirror only	Room caches Firestore data for offline reads. It is not a primary write target. Writes go to Firestore first.

4. System Design

4.1 Architecture Overview

FinPilot enforces strict MVVM (Model-View-ViewModel) architecture with a Repository pattern mediating between the ViewModel layer and data sources. The four layers are:

Layer	Responsibility	Key Files
UI (View)	Composable functions; observe StateFlow from ViewModel via collectAsState(); emit user events to ViewModel; NO business logic	DashboardScreen, IncomeScreen, ExpenseScreen, GoalTrackerScreen, AuthScreens
ViewModel	Holds and transforms UI state as StateFlow; executes use-cases via Repository; uses viewModelScope for coroutines; never references Context or View	DashboardViewModel, GoalViewModel, IncomeViewModel, ExpenseViewModel, AuthViewModel
Repository	Single source of truth; abstracts Firestore and Room; manages real-time listeners; handles offline sync; all methods are suspend or Flow-based	IncomeRepository, ExpenseRepository, GoalRepository, AuthRepository, FinanceRepository
Model / Data	Pure data classes (IncomeEntry, ExpenseEntry, Goal, FreelanceProject, UserProfile); Firestore documents; Room entities; shared between Firestore and Room via @Entity annotation	data/model/*.kt, data/local/entities/*.kt

4.2 MVVM Architecture Diagram

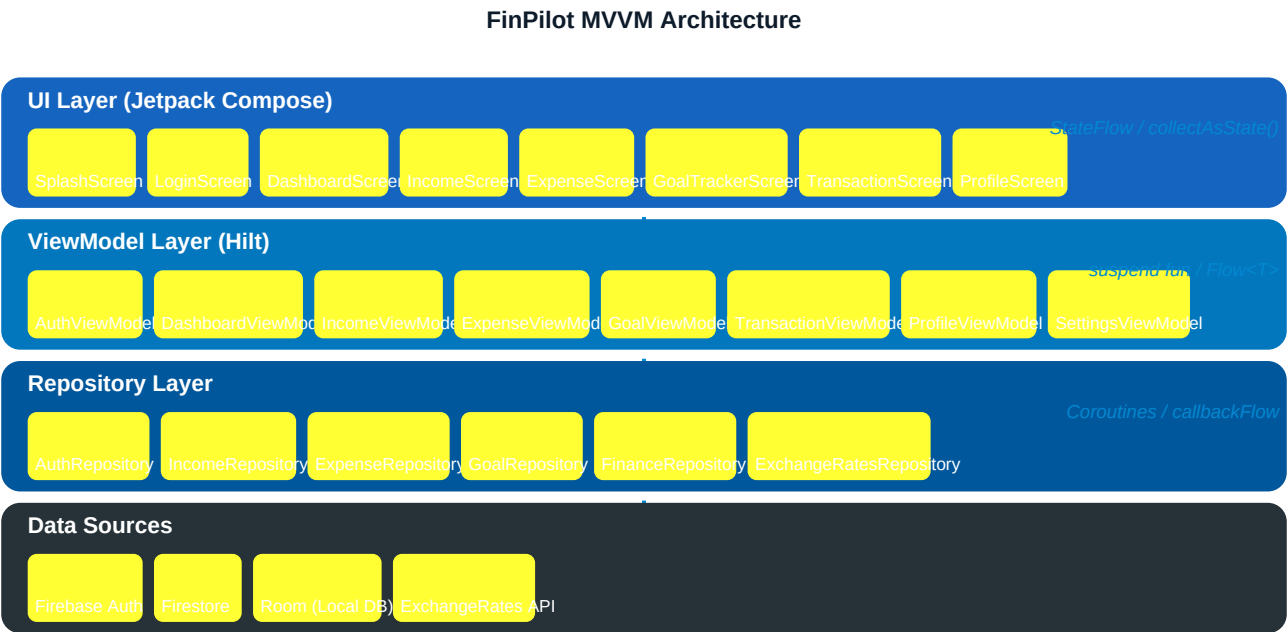


Figure 1: FinPilot MVVM Architecture — four strict layers with unidirectional data flow

4.3 Navigation Flow

FinPilot uses a single-activity architecture with Navigation Compose. All screens are composable destinations registered in FinPilotNavGraph. NavRoutes is a sealed class defining typed routes. Auth screens use popUpTo to clear the back stack on login/logout so the user cannot back-navigate into auth after signing in.

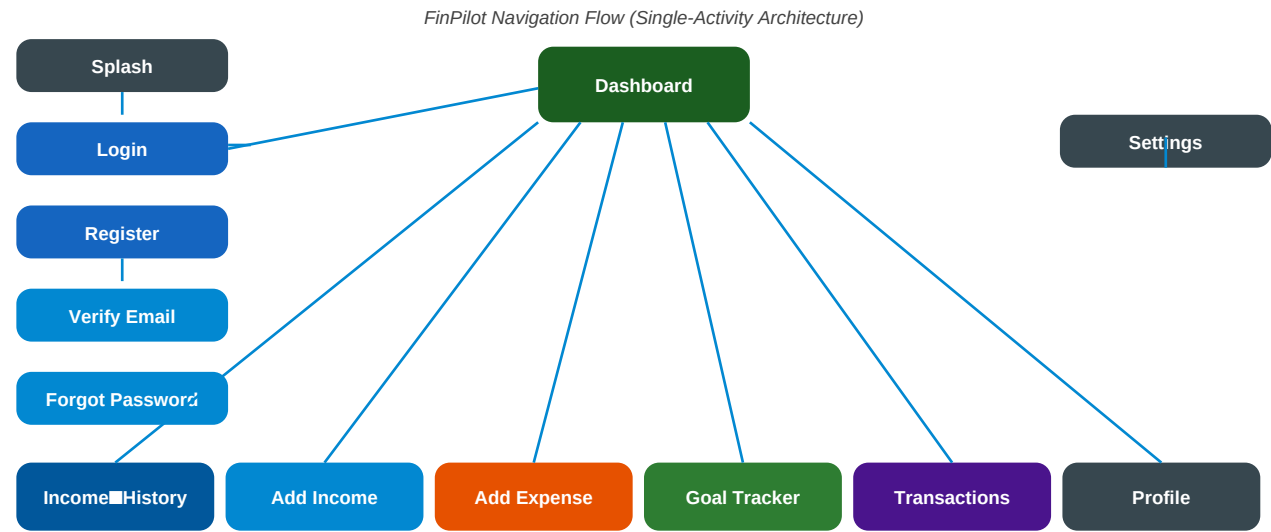


Figure 2: FinPilot Navigation Flow — authentication guard, main flow, and back-stack management

Route	Screen	Notes
splash	SplashScreen	Auth check; routes to Login or Dashboard
auth/login	LoginScreen	Email/password + Google Sign-In
auth/register	RegisterScreen	Create account; sends email verification
auth/verify_email	VerifyEmailScreen	Awaits verification; resend option
auth/forgot_password	ForgotPasswordScreen	Password reset via Firebase
dashboard	DashboardScreen	Home — chart-first financial overview
income	IncomeScreen	Paginated income history (15/page)
income/add	AddIncomeScreen	Add new income with source/currency selection
expense/add	ExpenseScreen	3-tap expense entry with category chips
goal/{goalId}	GoalTrackerScreen	Goal pager, savings log chart, log/withdraw
transactions	TransactionScreen	Unified income + expense history
profile	ProfileScreen	User settings, notification prefs, income sources
profile/settings	SettingsScreen	Theme, biometrics, cloud sync, data export

4.4 Repository Pattern

Each repository is annotated with `@Singleton` and injected via Hilt. The pattern ensures the ViewModel never directly touches Firestore or Room. Three repository classes handle the core data concerns:

Repository	Pattern	Description
IncomeRepository	Paginated load + MutableStateFlow	Loads income in pages of 15 from Firestore ordered by date DESC. Maintains lastDocument cursor for startAfter() pagination. Auth state listener auto-resets on sign-in/sign-out.
ExpenseRepository	Paginated load + MutableStateFlow	Same pagination pattern as IncomeRepository. Expenses ordered by date DESC. Singleton ensures single source of truth across all screens.
GoalRepository	Real-time Firestore listener	Uses addSnapshotListener for real-time goal updates. Maintains savingsLogs subcollection with 6-month windowed queries. Goals update immediately on the dashboard when Kavindu logs a savings event.
AuthRepository	StateFlow	Wraps FirebaseAuth with addAuthStateListener. Exposes currentUser as StateFlow. Handles email verification flow, Google Sign-In, password reset, and account deletion with mapped error messages.
FinanceRepository	callbackFlow + Room sync	Provides month-scoped queries (getIncomeByMonth, getExpensesByMonth) with real-time Firestore listeners and write-through Room caching for offline support.

5. Database Design

5.1 Firestore Schema Overview

Firestore is a document-oriented NoSQL database. The schema is designed around Kavindu's query patterns — what he needs to see on the Dashboard, what he filters by month, and what he aggregates by category. All data is user-scoped under `users/{uid}` to enforce data isolation.

Schema Pattern	<code>users/{uid} → income/{incomeId} expenses/{expenseId} goals/{goalId} goals/{goalId}/savingsLogs/{logId} freelanceProjects/{projectId}</code>
----------------	---

5.2 Collections & Documents

5.2.1 `users/{uid}`

Field	Type	Description
<code>uid</code>	String	Firebase Auth UID — matches document ID
<code>displayName</code>	String?	User's display name; set at registration
<code>email</code>	String	Email used for authentication
<code>baseCurrency</code>	String	Always 'LKR' for Kavindu; supports future multi-currency profiles
<code>createdAt</code>	Timestamp	Account creation time

5.2.2 `users/{uid}/income/{incomeId}`

Field	Type	Description / Justification
<code>source</code>	String	SALARY FREELANCE ADSENSE CRYPTO OTHER — enables income breakdown by channel on dashboard
<code>amountOriginal</code>	Number	Raw amount in original currency — preserves transparency ('12 USD received')
<code>currencyOriginal</code>	String	LKR USD USDT ETH — per-entry currency; solves the multi-currency income problem
<code>amountLKR</code>	Number	Normalised LKR equivalent — the single field used in all aggregations; removes runtime FX conversion
<code>exchangeRate</code>	Number	Rate applied at time of entry — creates implicit FX audit trail; honest about uncertainty
<code>date</code>	Timestamp	Actual date of receipt (not expected); enables month-scoped queries
<code>label</code>	String?	Human reference e.g. 'React build for Cinnamon Hotel' — prevents payment-project confusion
<code>type</code>	String	RECURRING ONE_OFF VARIABLE — enables salary vs freelance vs passive distinction
<code>projectRef</code>	String?	Optional FK to <code>freelanceProjects/{id}</code> — enables project-payment reconciliation

5.2.3 `users/{uid}/expenses/{expenseId}`

Field	Type	Description / Justification
<code>amount</code>	Number	Always in LKR — expenses are local; no FX needed in most cases

category	String	HOUSING FOOD TRANSPORT SUBSCRIPTIONS ENTERTAINMENT HEALTH OTHER — enforced at entry; enables donut chart
subCategory	String?	e.g. 'UberEats', 'PickMe', 'Gym' — optional second-level labelling for Kavindu's specific channels
paymentMethod	String	CARD CASH BANK_TRANSFER AUTO_DEBIT — captures all 4 channels missed by Kavindu's bank app
date	Timestamp	Actual date of spend; enables month-range queries
note	String?	Optional description — never required; friction would kill adherence
isRecurring	Boolean	true for rent, gym, subscriptions — enables committed/discretionary ratio computation
tags	Array	Flexible user-defined tags — future-proofing for personal filtering without schema changes

5.2.4 users/{uid}/goals/{goalId}

Field	Type	Description
title	String	e.g. 'MacBook Pro M4' — user-defined label
targetAmount	Number	490,000 LKR for Kavindu's MacBook goal
currentAmount	Number	Updated via logSavings() and withdrawSavings(); explicit savings events only
deadline	Timestamp ?	Target purchase date; drives monthlyRequired computation
monthlyRequired	Number	$(\text{targetAmount} - \text{currentAmount}) / \text{monthsRemaining}$ — computed and stored for offline access
isActive	Boolean	Supports multiple concurrent goals; dashboard shows first active goal

5.2.5 users/{uid}/goals/{goalId}/savingsLogs/{logId}

Field	Type	Description
amount	Number	Amount saved in this event
timestamp	Timestamp	When the savings was logged — used for 6-month aggregation in bar chart

5.2.6 users/{uid}/freelanceProjects/{projectId}

Field	Type	Description
clientName	String	Client organisation name
projectTitle	String	Project description e.g. 'WordPress site for Lanka Tiles'
agreedAmount	Number	Total agreed contract value in LKR
paidAmount	Number	Amount received so far — $\text{agreedAmount} - \text{paidAmount} = \text{outstanding}$
status	String	OPEN COMPLETED CANCELLED
entries	Array	List of income entry IDs linked to this project — enables payment reconciliation

5.3 Multi-Currency Handling Strategy

All aggregation and reporting uses LKR as the base currency. The strategy of storing both original currency values and LKR equivalents provides three advantages:

- Aggregation queries only need to sum the amountLKR field — no runtime FX conversion is required, simplifying all dashboard calculations.
- The original amount is preserved for transparency — Kavindu can see that his AdSense earned \$12 which was recorded as LKR 3,840 at 320 LKR/USD.
- Exchange rate history is implicitly maintained in income records, useful for year-end financial review and tax purposes.

Exchange Rate Flow	1. User opens Add Income screen for AdSense payment 2. Selects currency: USD, enters amountOriginal: 12.00 3. ExchangeRatesRepository fetches reference rate (320 LKR/USD) 4. amountLKR pre-calculated as 3,840 — user confirms or adjusts 5. Both 12.00 USD and 3,840 LKR stored in Firestore 6. Dashboard sums amountLKR field only — no FX at query time
---------------------------	---

5.4 Indexing Strategy

Firestore requires composite indexes for multi-field queries. The following indexes are required for FinPilot's core queries. These must be created in the Firebase Console or via firestore.indexes.json:

Collection	Fields Indexed	Query Purpose	Direction
income	userId ASC + date ASC/DESC	Dashboard income this month; paginated history	Composite
income	userId ASC + source ASC + date ASC	Income source breakdown for dashboard bar chart	Composite
income	userId ASC + type ASC + date ASC	Recurring vs one-off income analysis	Composite
expenses	userId ASC + date ASC/DESC	Dashboard expenses this month; paginated history	Composite
expenses	userId ASC + category ASC + date ASC	Category breakdown for donut chart	Composite
expenses	userId ASC + isRecurring ASC + date ASC	Committed vs discretionary split computation	Composite
goals	userId ASC + isActive ASC	Active goals for dashboard display	Composite
savingsLogs	timestamp ASC	6-month savings history chart	Single-field

5.5 Firestore Security Rules

Security rules enforce that each user can only read and write their own documents. The rules are user-scoped at the collection level:

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    // User profile – read/write own document only
    match /users/{userId} {
```

```
allow read, write: if request.auth != null
&& request.auth.uid == userId;

// Income subcollection
match /income/{incomeId} {
allow read, write: if request.auth != null
&& request.auth.uid == userId;
}

// Expenses subcollection
match /expenses/{expenseId} {
allow read, write: if request.auth != null
&& request.auth.uid == userId;
}

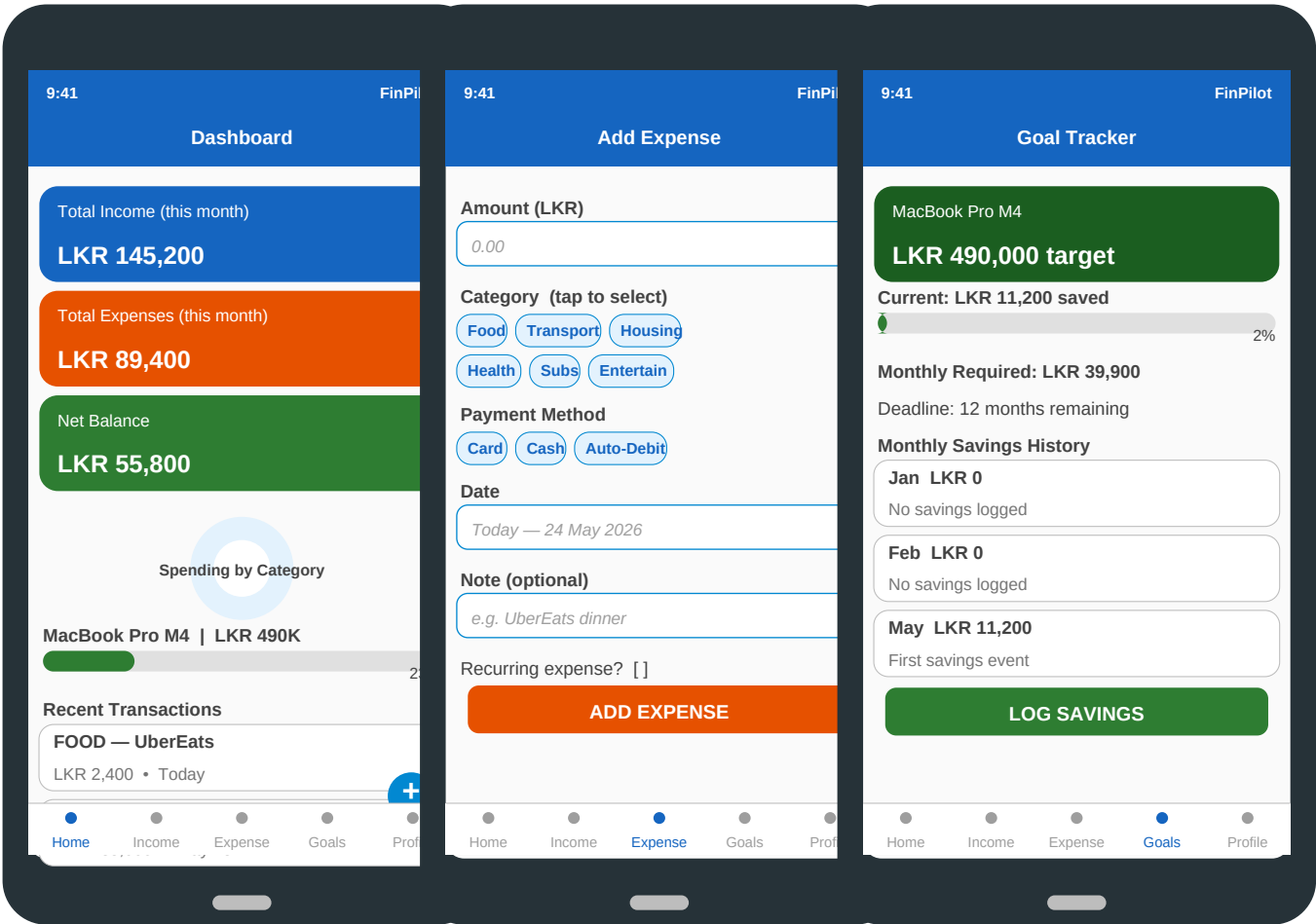
// Goals and savings logs subcollections
match /goals/{goalId} {
allow read, write: if request.auth != null
&& request.auth.uid == userId;
match /savingsLogs/{logId} {
allow read, write: if request.auth != null
&& request.auth.uid == userId;
}
}

// Freelance projects subcollection
match /freelanceProjects/{projectId} {
allow read, write: if request.auth != null
&& request.auth.uid == userId;
}
}
}
```

The rules enforce: (1) authentication is required for all operations, (2) users can only access documents within their own users/{uid} namespace, (3) subcollections inherit the same uid check from the parent document path. No cross-user data access is possible even if a document ID is known.

6. UI/UX Design — Annotated Wireframes

The following wireframes represent the five most critical screens in Kavindu's daily experience. Each is annotated with UX reasoning derived directly from the Phase One analysis. The visual design uses Material Design 3 with a dark/teal colour scheme implemented via Jetpack Compose theming.



Wireframe 1: Dashboard Screen

Wireframe 2: Add Expense Screen

Wireframe 3: Goal Tracker

Wireframe 1 — Dashboard Screen Annotations

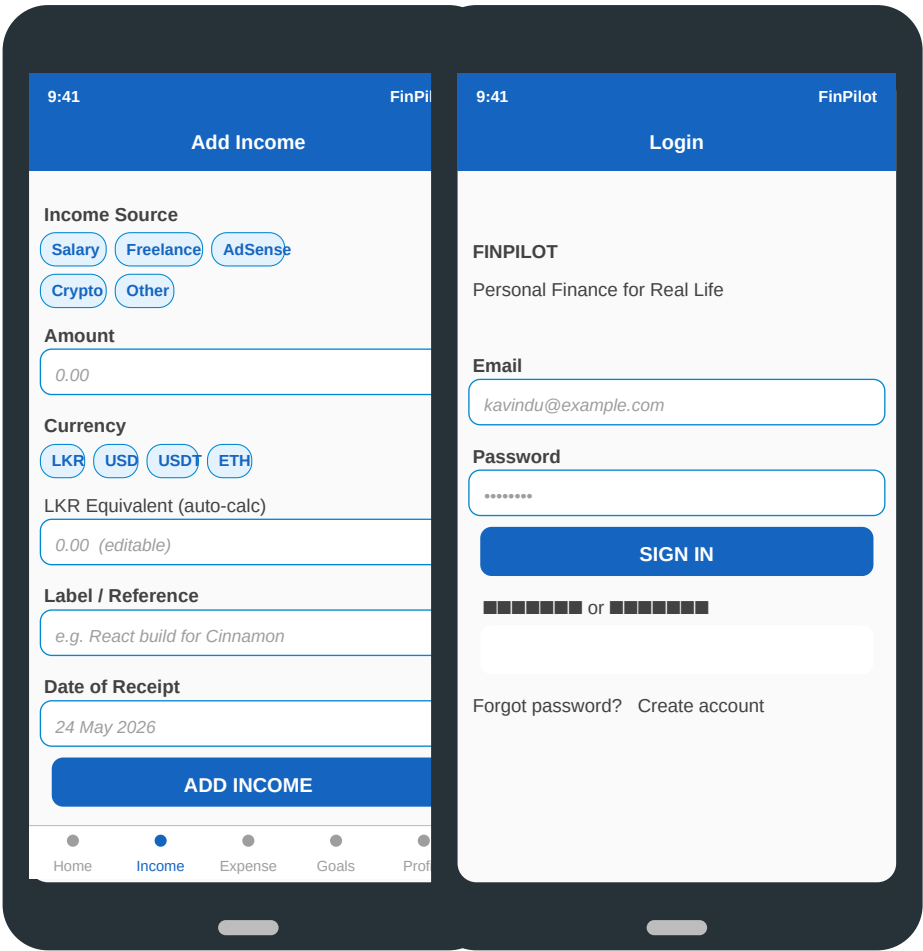
UI Element	UX Decision / Justification
Three summary cards (Income, Expenses, Net)	First thing Kavindu sees. The bank balance was meaningless; this is a curated financial summary. Direct response to 'no visual feedback' root cause.
Spending donut chart (always visible)	Surfaces the LKR 24,000 coffee discovery automatically. No navigation required. This is the insight Kavindu said 'would be immediately clarifying'.
Goal progress bar with monthly required	MacBook goal is visible every time the app opens. monthlyRequired updates in real time. Goal is never abstract again.
Recent transactions (5 items)	Quick scan without opening Transaction screen. Shows income and expenses interleaved.
FAB (+) for quick expense	Always accessible from home. Lowest-friction entry point into expense logging.

Wireframe 2 — Add Expense Screen Annotations

UI Element	UX Decision / Justification
Category as chip row (single tap)	Category is enforced but effortless. Never a typed field — a typed category would kill adherence. Direct response to UX root cause #1 (high friction).
Date pre-filled to today	The common case is entering an expense right after it happens. Requiring a date selection every time adds friction for zero gain.
Note field is optional	An optional note field respects that Kavindu is impulsive — he may not remember what the expense was for and should not be blocked from recording the amount.
isRecurring toggle	Critical for the committed/discretionary ratio. Rent, gym, subscriptions should be marked here — FinPilot then computes what proportion of income is already allocated.
No confirmation dialog	Confirmation dialogs add one more tap to every entry. The undo pattern (future enhancement) is preferable to friction at the confirmation stage.

Wireframe 3 — Goal Tracker Screen Annotations

UI Element	UX Decision / Justification
Large goal card with target at top	The goal is the hero element of this screen. The LKR 490,000 target is always prominent — it anchors the user's motivation.
Progress bar with percentage and absolute amounts	Both visual (bar) and numerical (LKR X saved) representations. A bar without numbers is incomplete; numbers without a bar lack intuitive impact.
Monthly Required (computed field)	$(\text{targetAmount} - \text{currentAmount}) / \text{monthsRemaining}$ — the answer to Kavindu's question: 'What do I need to save per month?' Never available in any previous tool.
Monthly savings history bar chart	6-month trend from savingsLogs subcollection. Shows whether the habit is forming. Months with no logs show LKR 0 — never broken.
Log Savings button	Explicit savings events only. Kavindu must designate money as 'saved' — it does not auto-deduct from expenses. This creates deliberate saving behaviour.



Wireframe 4: Add Income Screen

Wireframe 5: Login Screen

Wireframe 4 — Add Income Screen Annotations

UI Element	UX Decision / Justification
Source type chip row (SALARY, FREELANCE, ADSENSE, CRYPTO)	The four known channels are first-class citizens, not buried in a dropdown. Each is a single tap. Custom 'Other' available for new sources.
Currency selector per income entry	The root cause of AdSense and crypto tracking failure: no tool ever supported multi-currency income. This selector directly fixes that.
LKR Equivalent auto-calculated but editable	ExchangeRatesRepository pre-populates amountLKR from reference rate. Kavindu can override with the actual rate at time of transfer — more honest than a fixed rate.
Label / Reference field	For freelance: 'React build for Cinnamon Hotel'. For crypto: 'USDT withdrawal Jan'. This field prevents the payment-project confusion Kavindu has experienced twice.

Wireframe 5 — Login Screen Annotations

UI Element	UX Decision / Justification
Email/password (primary)	Firebase Authentication with email/password. Mandatory per assignment requirements.
Google Sign-In (secondary)	Bonus credit. Reduces onboarding friction — Kavindu's Google account is already trusted. Uses Firebase GoogleAuthProvider.

Email verification flow	Registration sends verification email; user redirected to VerifyEmailScreen. Login blocked until email is verified. Prevents abandoned unverified accounts from polluting Firestore.
Forgot Password link	Firebase sendPasswordResetEmail. Full error mapping for all FirebaseAuthException codes provides user-friendly messages instead of raw Firebase error strings.

7. Design Justifications

This section explains, for each major design decision, why it was made in direct response to Kavindu's specific situation and the documented failures of his prior tools.

7.1 Why FinPilot Avoids Kavindu's Previous Failures

Previous Failure Pattern	FinPilot Architectural Decision	Outcome for Kavindu
Google Sheet abandoned after 13 days due to manual overhead	3-tap expense entry with pre-filled defaults (today's date, CARD, LKR). No entry is 'incomplete' enough to reject — amount + category is sufficient.	Entry overhead reduced to under 10 seconds. No formulae to break. Mobile-native design matches how Kavindu actually lives.
Budget app installed 9 days — USD only, single income model	Income model built around 4 source types with per-entry currency selection. LKR is the aggregate base; USD/USDT/ETH preserved in original form.	Every income channel Kavindu uses is a first-class citizen. No workarounds. No 'custom category hacks'.
Notes app unusable after 4 days — cognitive overload from raw text	Chart-first dashboard. Spending donut, income bar, goal progress — all visual and above the fold. Text lists are secondary.	Kavindu sees a picture, not a table. The LKR 24,000 coffee discovery would surface automatically in week one.
Bank app: card-only, miscategorised transactions	All payment methods (CARD, CASH, BANK_TRANSFER, AUTO_DEBIT) supported per expense entry. User controls categorisation — no automatic miscategorisation.	Kavindu's gym auto-debit, PickMe cash payments, and card transactions all exist in one aggregated view for the first time.
All tools: no recovery path after gaps in entries	Month views display whatever data exists. Empty states are positive and actionable, not punitive. Partial data produces valid (if incomplete) charts.	Missing a week does not break the system. The app is equally useful on day 3 and day 30 regardless of consistency.

7.2 Why the Expense Flow Is Low-Friction

The expense flow is FinPilot's most critical UX surface. Kavindu's core failure mode is abandonment after a gap — and the threshold for abandonment is directly correlated to friction. FinPilot reduces this friction through four deliberate decisions:

- **Pre-filled defaults:** Date = today. Payment method = CARD. Currency = LKR. These are the correct values for >80% of Kavindu's expenses. Pre-filling them removes 3 interactions from every common entry.
- **Category as chips, never a typed field:** Typing a category creates a free-text field that is unsearchable, inconsistently named, and visually dead. Single-tap chips create consistent, queryable categories that power the donut chart.
- **Floating action button always visible:** The add-expense trigger is never buried in a menu. It is one tap from any screen. This matches Kavindu's impulsive spending pattern — he needs to record right after the purchase, not after navigating.
- **Optional fields are genuinely optional:** Note, sub-category, and tags will never block entry. A record with only amount + category is analytically useful for the donut chart, the committed/discretionary ratio, and month comparisons.

7.3 Why the Dashboard Helps Kavindu Specifically

Kavindu's bank app shows him a transaction list. His Google Sheet showed him numbers. Neither produced a picture. The dashboard is the product's answer to this failure. Three decisions make it work for someone with his specific psychology:

- **Charts above the fold, lists below:** The spending donut, income bars, and goal progress card are visible on first render. Kavindu does not need to scroll to get insight. If he opens the app 10 times and leaves immediately, he still has seen his financial picture 10 times.
- **Month-over-month comparison:** The DashboardViewModel computes income, expenses, and net balance for both the current month and the previous month, producing a percentage change. This gives Kavindu the trend signal he has never had — 'Am I doing better or worse than last month?'
- **Did you know? contextual insight:** The DashboardViewModel generates a contextual insight string based on category concentration, fixed cost percentage, or average daily spend. This is not gamification — it is the type of observation that Kavindu's 3-month statement review surfaced manually (LKR 24,000 on coffee) now surfaced automatically on the first week.

7.4 Why the Goal Tracking Design Is Motivating

Kavindu's MacBook goal failed in Google Sheets because the goal was static — a number in a cell that never connected to his daily behaviour. FinPilot's goal system is built around three principles:

- **Real-time monthly requirement:** $\text{monthlyRequired} = (\text{targetAmount} - \text{currentAmount}) / \text{monthsRemaining}$. Stored in Firestore; recomputed whenever `goalRepository.upsertGoal()` is called. Every savings deposit makes the monthly requirement drop. Every month that passes without saving makes it rise. The goal is never static.
- **Explicit savings as a deliberate act:** Savings events are logged deliberately (`logSavings()`), not auto-calculated from surpluses. This creates intentional financial behaviour — the act of logging a savings event is itself a commitment. The projected surplus (income – expenses) is shown as a secondary figure to show Kavindu the gap between what he could theoretically save and what he actually designates as saved.
- **Goal visible on dashboard every session:** The GoalRepository uses an `addSnapshotListener`, so the goal card on the dashboard updates in real time. Kavindu never needs to navigate to the Goal screen to know his status. The goal is the first context he sees when opening the app.
- **Multi-goal pager:** GoalTrackerScreen supports a horizontal pager of multiple goals. When Kavindu achieves the MacBook goal, he can start a new one without losing history. Goals that are completed remain in the pager as motivation.

7.5 Architecture Justification

Decision	Justification
Hilt DI over manual injection	Removes the anti-pattern of constructing ViewModel dependencies inside Composables. All repositories are <code>@Singleton</code> injected — single source of truth across all screens.
StateFlow over LiveData	StateFlow is lifecycle-aware and flows natively in Kotlin coroutines. <code>collectAsState()</code> in Compose is idiomatic. LiveData would require additional boilerplate in a pure-Kotlin codebase.
Paginated load (IncomeRepository, ExpenseRepository)	Kavindu's transaction history will grow indefinitely. Loading all records at once on a Spark plan is inefficient and slow. Pages of 15 with <code>startAfter()</code> cursor pagination keep initial load fast and predictable.
GoalRepository with real-time listener	Goals must update on the Dashboard without user interaction. A one-shot query would miss updates made from the Goal screen. <code>addSnapshotListener</code> ensures the dashboard goal card is always current.

Room as offline mirror of Firestore	The assignment prohibits blank screens on network failure. Room mirrors Firestore data locally. When Firestore is unavailable, the UI reads from Room. Writes are queued for sync. This directly prevents the failure mode Kavindu experienced — a tool that shows nothing useful when offline is abandoned.
Email verification before login	Registration sends verification, then signs out. Login is blocked until email is verified. This prevents abandoned unverified accounts from accumulating in Firestore and ensures Kavindu's data is tied to an account he actually controls.

8. Conclusion

8.1 Summary

FinPilot is a purpose-built personal finance management application designed from the ground up for Kavindu Silva's specific combination of complexity, irregularity, and aspiration. Every design decision traces back to a documented failure in his financial history or a specific need surfaced by the Phase One analysis.

The application implements strict MVVM architecture with Hilt dependency injection, Firebase Authentication (email/password + Google Sign-In), Firebase Firestore with real-time listeners, Room local caching for offline support, and Jetpack Compose for all UI. It runs on Android API 26 and above, targeting API 36.

The five core problems it solves — multi-source multi-currency income tracking, low-friction expense categorisation, real-time goal progress, visual financial awareness, and offline resilience — are precisely the five failures that every previous tool Kavindu used was unable to address.

8.2 Expected Impact on Kavindu's Financial Behaviour

Behaviour Change	Mechanism
Kavindu accurately knows his monthly income for the first time	Four income sources tracked individually with LKR normalisation. The gap between his LKR 160,000–210,000 estimate and his actual income is made visible.
Kavindu stops being shocked by his spending	Category donut chart on the dashboard surfaces concentration (e.g. food, dining, transport) automatically — the LKR 24,000 coffee discovery becomes a weekly check, not a 3-month surprise.
Kavindu knows exactly what he needs to save per month for his MacBook	monthlyRequired is computed and prominently displayed. The goal is no longer abstract — it is LKR 39,900/month at current savings, update immediately as he saves.
Kavindu stops abandoning tracking after gaps	3-tap expense entry and always-useful partial data means a missed week costs nothing. The app is equally valuable after returning from a gap.
Kavindu sees the committed/discretionary split for the first time	isRecurring flag on expenses enables FinPilot to show: 'LKR 49,000 of your income is already committed before discretionary spending begins.' This reframes every purchase decision.

Assignment Criterion Alignment

Problem Analysis (20 marks) — Phase One analysis (Section 2) derives all requirements from Kavindu's scenario with explicit traceability. | Architecture (20 marks) — Strict MVVM, Hilt, StateFlow, paginated repositories documented in Sections 4 and 7. | Compose UI (15 marks) — Five annotated wireframes in Section 6 with UX justification. | Firebase (15 marks) — Firestore schema, indexing, security rules in Section 5. | Documentation (10 marks) — Requirements, schema, MVVM diagram, wireframes all present.

If this submission makes the panel genuinely believe that Kavindu would open this application daily, that it would change how he understands his money, and that he would still be using it six months from now — we will have done this assignment justice.

— FinPilot Team: Kaveen, Venuja, Vinuki, Sahaji